Data challenge 2026 Defective columns

Cecilia Frimpong Jessica Bibby

Lyo Saillot Renyi Yang

Yano Salum Schettini

June 2026

1 Introduction

The objective is to identify and fix defective vertical columns in infrared image sequences (16-bit PNG). These defects stem from readout circuit problems (OpAmp sensitivity) and manifest mainly in three forms:

- **Noisy columns:** Persistently exhibit higher temporal noise compared to the background.
- **Blinking columns:** Show sudden digital level shifts, either high or low frequency, over time.
- **Fragmented blinking columns:** Exhibit rapid temporal and spatial changes, with events appearing or disappearing per frame, possibly multiple times per column.

The results should achieve high F1-score (detection) and a low RMSE (correction) on hidden test data.

Final score (max = 1):

$$\begin{aligned}\text{FinalScore} = {} & 0.34 \times F1 \\ & + 0.33 \times \left(1 - \min\left(\frac{\text{RMSE}_{\text{defective}}}{40}, 1\right)\right) \\ & + 0.33 \times \left(1 - \min\left(\frac{\text{RMSE}_{\text{ok}}}{40}, 1\right)\right)\end{aligned}$$

We present an evaluation of multiple detection strategies and propose a robust 1D correction pipeline. Our final solution achieves strong detection performance and high-quality restoration across different sensors and defect types.

2 Proposed Methodology

2.1 Detector 1: Difference in Median

This method uses the difference between consecutive columns to detect noisy columns. It assumes that defective columns have significantly higher or lower median pixel value than their neighbours. The method doesn't use the temporal axis, and doesn't address fragmented defective columns. The algorithm is applied to each frame independently.

Algorithm steps

1. Compute the median of each columns using `median = np.median(img, axis=0)`
2. Compute the difference and take its absolute value using `diff = np.abs(np.diff(median))`
3. Use a threshold of `threshold = 2.5 * np.mean(diff)` to detect noisy columns where `diff > threshold`

The difference `diff` of an array `arr` is computed for each index `i` using `diff[i] = arr[i + 1] - arr[i]`.

Results

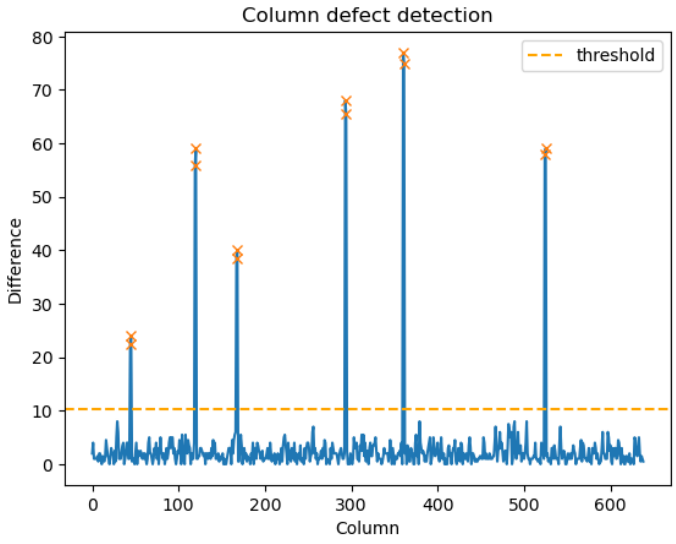

Figure 1: Example visualisation of detected columns using difference in median. (VGA Sequence 1 columns 3 frame 00)

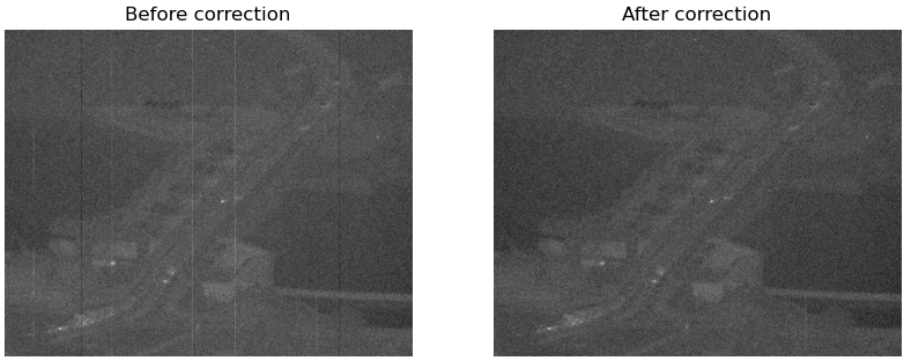

Figure 2: Example output of a frame (VGA Sequence 1 columns 3 frame 00) using difference in median

This method gives an average F1 score of 0.26, with 0.002 for HD, 0.003 for SXGA and 0.77 for VGA.
While this algorithm performs correctly with the VGA sensor, it doesn't work well on the other sensors. As the difference is computed on consecutive columns, it detects changes in consecutive columns. While this works on isolated defective columns, it doesn't work when there are consecutive defective columns, which HD and SXGA sensors have, unlike VGA.
Therefore, we modified this method in a later detector to account for those columns.

2.2 Detector 2: Temporal standard deviation

The method computes the temporal standard deviation per pixel in each column, aggregates it, and applies a threshold to identify defective columns. Well-functioning columns have pixel values that vary primarily due to scene content and sensor noise, whereas noisy columns exhibit additional independent variation, raising their standard deviation across frames. Mathematically, for column c, the pixel-wise standard deviation is calculated as

$$\sigma(p,c) = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(I_t(p,c) - \mu(p,c)\right)^2}$$

where $I_t(p,c)$ is the intensity at pixel p in column c at time t, and $\mu(p,c)$ is the temporal mean. A column is flagged as defective if the average (or median) of $\sigma(p,c)$ across rows p exceeds a statistical threshold.

Algorithm steps

1. **Stack loading:** Read all 16-bit PNG frames of a sequence into a 3-D NumPy array of shape (T, H, W).

2. **Temporal statistics:** Compute temporal standard deviation per pixel: `temporal_std = np.std(stack, axis=0)`

3. **Column Aggregation:** Reduce the temporal standard deviation map to a 1-D vector: `column_noise = np.mean(temporal_std, axis=0)` (or `np.median(temporal_std, axis=0)` for increased robustness).

4. **Thresholding:** Compute the mean and standard deviation of `column_noise`; flag columns satisfying `column_noise > mean + k * std`, where `k` is tuned to `3.0`.

5. **Correction:** Replace flagged columns by median (or mean) interpolation from neighbouring good columns

Implementation highlights

The reference implementation, built with Python, OpenCV, and NumPy, follows the described algorithm. Below are key code snippets, simplified for clarity:

```
k = 3.0    # or 2.6, 2.7, etc. tuned
temporal_std = np.std(stack, axis=0)
column_noise = np.mean(temporal_std, axis=0)
mean_noise = np.mean(column_noise)
std_noise = np.std(column_noise)
threshold = mean_noise + k * std_noise
bad_columns = np.where(column_noise > threshold)[0]
```

The correction is simple interpolation between left and right neighbours:

```
corrected[:, c] = 0.5 * (
    img[:, c - 1] +
    img[:, c + 1]
)
```

Results

The evaluation using the 2metrics file is moderate, with an F1 score of approximately 0.5. The detection of more prominent noisy columns is achieved; however, flickering and fragmented columns are ignored. In summary, although employing the temporal standard deviation for defect detection is effective in identifying some faulty columns, the moderate F1 score suggests that alternative detection and correction strategies should also be explored.

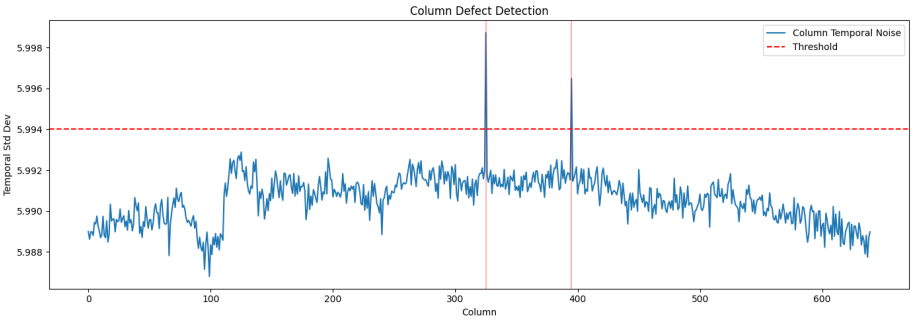

Figure 3: Example visualisation of detected columns using temporal std (VGA)

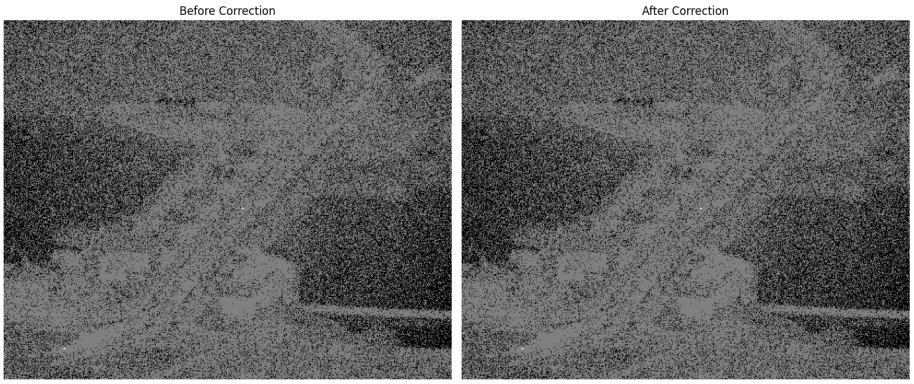

Figure 4: Example output of a single corrected frame (VGA) using temporal std detection

2.3 Detector 3: Isolation Forest

This method employs an unsupervised anomaly detection framework based on the Isolation Forest algorithm to identify defective columns in infrared image sequences. The approach assumes that defective columns exhibit statistical characteristics that differ significantly from nominal background columns. By leveraging Isolation Forest, anomaly localization can be automated using engineered features, eliminating the need for manually selected thresholds.

Once a defective column is detected, correction is performed through interpolation using the immediate horizontal neighbouring columns. This provides a baseline proof-of-concept for automated defect detection and restoration.

To characterize each image column, four statistical descriptors are extracted across the image sequence:

- **Temporal Mean Intensity**: The average brightness of a column over time.

- **Temporal Variability**: The fluctuation of intensity values across successive frames.
- **Spatial Standard Deviation**: The internal pixel-wise variance within a column.
- **Differential Context**: The contrast between a target column and its adjacent neighbouring columns.

These descriptors are combined into a feature matrix where each image column is represented as an observation vector. The Isolation Forest algorithm evaluates this feature space by recursively partitioning the data using random feature selections and split values. Since anomalous observations require fewer partitions to become isolated, they receive higher anomaly scores.

Columns whose anomaly scores exceed a predefined threshold are classified as defective.

Algorithm steps

1. Load all frames from the target infrared image sequence.
2. Extract column-wise spatial and temporal statistical features.
3. Construct a feature matrix from the extracted descriptors.
4. Train the Isolation Forest model on the resulting dataset.
5. Compute anomaly scores for each image column.
6. Classify columns as nominal or defective based on anomaly scores.
7. Validate detected defects against ground-truth annotations.
8. Correct defective columns using neighbouring-column interpolation.
9. Evaluate restoration quality using quantitative and qualitative metrics.

The pipeline supports automated feature extraction, anomaly detection, localization, and restoration without manual intervention. Performance was evaluated using precision, recall, and F1-score, together with qualitative inspection tools such as anomaly score plots, defect localization maps, and before-and-after restoration comparisons. The Isolation Forest baseline achieved an average score of approximately 0.49 in the evaluated VGA image sequences.

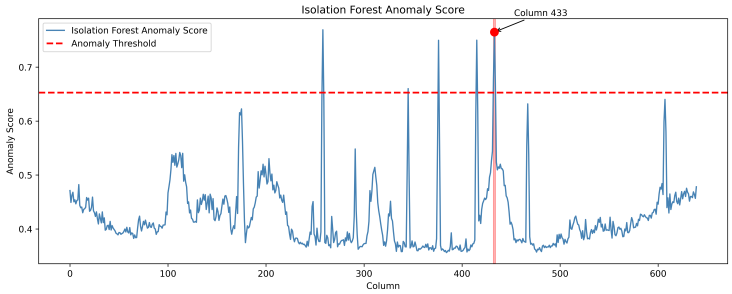

Figure 5: Defective Column Identification Using Isolation Forest. The anomaly score generated by the Isolation Forest model for each image column. Column 433 exceeded the anomaly threshold and was selected as a defective column.

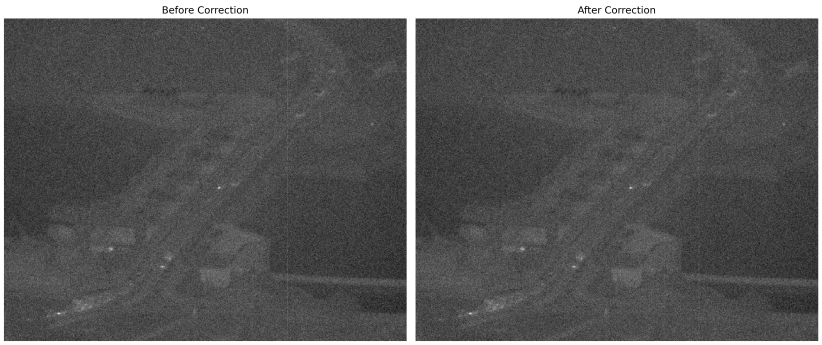

Figure 6: Before-and-After Comparison of Isolation Forest-Based Column Correction. Although the detected column was corrected using neighboring-column interpolation, the visual difference remained subtle because the defect occupied only a narrow region of the image.

2.4 Detector 4: Convolutional Neural Networks

To resolve highly non-linear spatiotemporal degradation patterns such as flickering and fragmented blinking columns that escape global statistical filtering we developed deep learning-based architectures. The task is formulated as a precise dense feature mapping problem over the spatial domain, exploring both 1D and 2D Convolutional Neural Network configurations.

2.4.1 1D-CNN Framework (Lightweight Alternative Option)

The initial deep-learning option models column defects by condensing the spatiotemporal cube into rapid 1D parallel sequences.

Feature Extraction and Efficiency:

For an image stack of shape $(Time, Height, Width)$, the spatial vertical height H is averaged out, generating a column-wise temporal signal of size $(Time, Width)$. Four distinct statistical metrics are computed per column across time: mean, standard deviation, maximum, and minimum, providing a condensed feature vector of shape $(4, Width)$.

Architecture:

The 1D-CNN maps the $(4, W)$ feature space to a 1D probability array of size W. The network consists of three cascaded `Conv1d` layers:

- Layer 1: Kernel size 5, padding 2, generating 16 channels, followed by 1D Batch Normalization and ReLU.
- Layer 2: Kernel size 3, padding 1, maintaining 16 channels, followed by Batch Normalization and ReLU.
- Layer 3: Kernel size 1, compressing the representation into a single channel, followed by a Sigmoid activation to output defect probabilities $P \in [0, 1]$.

Columns where $P > 0.5$ are formally classified as defective.

Pros and Operational Edge:

The architectural advantage of the 1D-CNN model lies in its exceptional computational speed and minimalist resource footprints. By shifting from a 3D spatiotemporal volume straight to 1D structural processing, it completely bypasses massive 2D matrix multiplications. The inference pipeline is highly accelerated, making it an excellent alternative candidate for real-time edge processing or low-latency systems. However, collapsing the height axis inevitably erases fine-grained vertical pixel structures, limiting its fine precision against short, fragmented blinking column pieces.

2.4.2 2D Asymmetric CNN Framework

To overcome the spatial limitations of the 1D approach, we developed a 2D-CNN that preserves the height dimension H, allowing the network to leverage localized vertical continuity.

2D Feature Extraction:

Instead of averaging over the height, the four temporal statistics (mean, std, max, min) are computed pixel-wise across the time axis T. This preserves the full spatial topology, resulting in an input feature tensor of shape $(4, H, W)$. Features are standardized spatially per channel.

Asymmetric Architecture:

Infrared column defects present a unique morphological signature: they are strongly correlated vertically but strictly isolated horizontally. Using standard square kernels (e.g., 7×7) would induce unnecessary cross-talk between healthy and defective adjacent columns. Therefore, we utilize *asymmetric convolutions*:

- Layer 1: `Conv2d` with a long vertical kernel size of $(7, 3)$ and padding $(3, 1)$ $\rightarrow$ 16 channels.
- Layer 2: `Conv2d` with a kernel size of $(5, 3)$ and padding $(2, 1) \rightarrow 16$ channels.
- Layer 3: 1×1 convolution mapped through a Sigmoid activation, yielding a 2D spatial probability mask.

Training and Data Augmentation:

The dataset consists of varying sensor resolutions (VGA, SXGA, HD). To unify the training pipeline and prevent overfitting, we dynamically sample 128×128 spatial patches (`RandomPatchDataset`) during training. The network is optimized using Binary Cross Entropy (BCE) loss and Adam optimizer (learning rate 0.001, weight decay $1e-4$).

Morphological Post-processing:

Direct thresholding of the CNN output mask is susceptible to isolated salt-and-pepper false positives. To robustly convert the 2D probability map into a final list of defective column indices, a morphological filtering pipeline is applied. First, a raw Boolean mask is generated using a confidence threshold (e.g., $P > 0.6$). Next, a Morphological Opening operation (`cv2.MORPH_OPEN`) is performed using an elongated vertical kernel of size $(9, 1)$. This geometric constraint effectively suppresses isolated noisy pixels while strictly preserving vertical defect fragments. Finally, any column containing at least one active pixel post-morphology (`mask.any(axis=0)`) is recorded as a defective column.

2.4.3 Experimental Evaluation and Hyperparameter Sweeps

The localization capabilities of the CNN methods were evaluated across varying multi-resolution arrays. This evaluation focuses exclusively on localized precision, recall, and final F1-scores.

Evaluation Under `Metrics v1.0`:

The 2D asymmetric network was jointly trained across the full multi-sensor dataset (VGA, SXGA, and HD). Under the `Metrics v1.0`, varying the hyperparameters and training strategies yielded the following aggregate F1-score benchmarks:

- **Baseline Strategy (VGA Limited):** Models restricted solely to VGA training samples reached a continuous average F1-score of **0.82**. This demonstrated a baseline capacity but showed limited transferability to the more complex structures.
- **Kernel Variant (5,1) Without Safety Bounds (Full 3-Set Mix):** Blending all resolutions and introducing a $(5, 1)$ kernel significantly improved generalization, pushing the baseline aggregate F1-score to **0.89**.
- **Kernel Variant (5,1) with Safety Threshold:** Add strict activation safety bounds yielded an F1-score of **0.85**.
- **Kernel Variant (7,1) with Safety Threshold:** Shifting to a wider $(7, 1)$ vertical receptive path under safety restrictions successfully recovered the localization trace, achieving an F1-score of **0.87**.
- **Kernel Variant (7,1) Without Safety Bounds:** Operating without safety bounding limits achieved an average F1-score of **0.87**.

Table 1: Evaluation Under `Metrics v1.0`

Strategy Configuration	Sensor	TP	FP	FN	Precision	Recall	F1-Score
(5,1) Without Safety Bounds	VGA	6.802	0.293	0.137	0.952	0.972	**0.959**
	SXGA	8.622	2.129	2.028	0.798	0.819	**0.800**
	HD	4.750	0.417	0.667	0.924	0.890	**0.905**
(5,1) with Safety Threshold	VGA	6.802	1.282	0.137	0.841	0.972	0.891
	SXGA	8.766	2.692	1.884	0.768	0.830	0.787
	HD	4.843	0.856	0.573	0.870	0.906	0.883
(7,1) with Safety Threshold	VGA	6.802	0.574	0.137	0.904	0.972	0.930
	SXGA	8.705	2.389	1.945	0.783	0.826	0.793
	HD	4.763	0.599	0.653	0.900	0.893	0.894
(7,1) Without Safety Bounds	VGA	6.802	0.575	0.137	0.904	0.972	0.930
	SXGA	8.705	2.389	1.945	0.783	0.826	0.793
	HD	4.764	0.599	0.653	0.900	0.894	0.894

Evaluation Under `Metrics v2.0`:

Due to some code problem, we get a new version of Metrics. The score is lower than expected, so we design new approaches to increase it.

To effectively isolate valid column defects under the revised discrete matching standard, extensive parameter sweeps were conducted. We systematically adjusted confidence thresholds alongside custom multi-scale column area constraints (minimum valid pixel heights).

This approach assigns distinct height thresholds tailored specifically to the HD, SXGA, and VGA sensors. We experimented with numerous combinations of these multi-scale parameters across the three sensor types (for instance, evaluating HD-SXGA-VGA `col_cnt` triplets such as 35-55-35, 35-70-35, and 35-120-35). While these comprehensive sweeps informed our strategy, only the optimal

hyperparameter configurations that yielded the peak performance are reported in the results below.

Lowering the activation threshold effectively boosts raw Recall, capturing subtle or low-frequency blinking frames, but inherently increases background false alarms (FP). Pairing this lowered threshold with multi-scale morphological area constraints successfully filters out the false positives while preserving the newly recovered true defects.

Table 2: Evaluation Under `Metrics v2.0`

Hyperparameter Configuration	Sensor	TP	FP	FN	Precision	Recall	F1-Score
Threshold 0.50 Basic Opening-Closing	VGA	42	20	18	0.6774	0.7000	0.6885
	SXGA	-	-	-	-	-	-
	HD	119	41	89	0.7438	0.5721	0.6467
Threshold 0.35 Column Area Filter	VGA	43	17	17	0.7167	0.7167	0.7167
	SXGA	157	103	75	0.6038	0.6767	0.6382
	HD	135	39	73	0.7759	0.6490	0.7068
Threshold 0.25 Multiscale Area Filter	VGA	44	14	16	0.7586	0.7333	**0.7458**
	SXGA	159	99	73	0.6163	0.6853	**0.6490**
	HD	141	21	67	0.8704	0.6779	**0.7622**
Threshold 0.15 Multiscale Area Filter	VGA	44	24	16	0.6471	0.7333	0.6875
	SXGA	-	-	-	-	-	-
	HD	16	10	0	0.6154	1.0000	0.7619

As shown in Table 2, systematically lowering the threshold improves true defect localization across the datasets. By relaxing the threshold to 0.25, raw False Negatives drop significantly. Although raw False Positives experience a spike, the custom multi-scale area filter effectively isolates and eliminates background noise, delivering the peak F1-score performance. Pushing the threshold excessively low (e.g., 0.15) introduces overwhelming background clutter, causing noise to bypass the morphological defense layer and severely degrading the final F1-score due to compromised precision.

Evaluation Under `Metrics v3.0`:

Due to some code problem again, we get another new version of Metrics. However, empirical verification confirmed that its performance plateaued around an average F1-score of ~ 0.5. Furthermore, it lacked the structural tracking stability exhibited by our optimized **final_version**. Consequently, this approach was formally discarded.

2.5 Detector 5: Local mean of medians

This method is a modified algorithm of the difference method to detect consecutive defective columns. It uses a uniform filter on the columns medians to compute a local mean, and then compute the difference between the local mean and the medians to detect the defective columns.

Algorithm steps

1. Compute the median of each columns using `median = np.median(img, axis=0)`
2. Compute the local mean with a uniform filter using `local_mean = uniform_filter1d(median, size=20)`
3. Compute the difference between the median and local mean, and take its absolute value using `ratio = np.abs(median - local_mean)`
4. Use a threshold of `threshold = 3 * np.std(ratio) + np.mean(ratio)` to detect noisy columns where `ratio > threshold`

Results

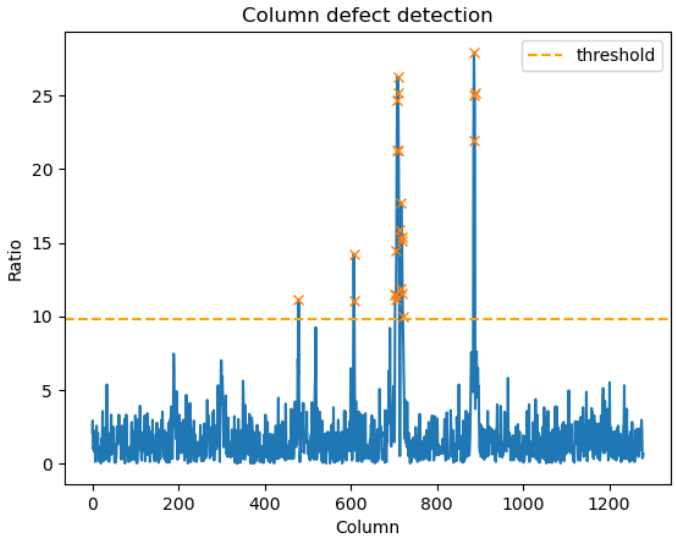

Figure 7: Example visualisation of detected columns using difference in median. (SXGA Sequence 2 columns 3 frame 00)

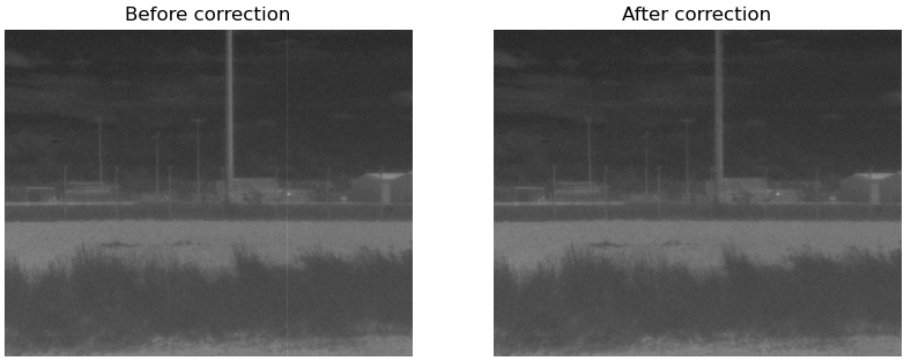

Figure 8: Example output of a frame (SXGA Sequence 2 columns 3 frame 00) using difference in median

This method gives an average F1 score of 0.57, with 0.51 for HD, 0.52 for SXGA and 0.70 for VGA.
We chose this method as our final version as it gave us the best final score.

2.6 Correction

Once defective columns are identified by the detection module, the correction step reconstructs the missing signal without degrading the underlying infrared image. We formulated this as a **signal decomposition problem**: rather than replacing defective pixel values directly, we estimate the additive stripe noise component S and subtract it from the raw signal, yielding:

$$X_{\text{corrected}} = X_{\text{raw}} - S \tag{1}$$

This formulation preserves all original infrared radiometric information outside the defective region, guaranteeing $\text{RMSE}_{\text{ok}} = 0$ by construction, since healthy pixels are never modified.

Final method: Side-Window Filter + 1D Gradient Domain Guided Image Filter.

Our final approach combines two techniques into a sequential pipeline applied independently to each defective column. The method is inspired by [1] and [2], adapted to the localized column defect setting.

The pipeline executes four sequential steps per defective column. First, a clean reference signal U is estimated by computing a distance-weighted average of the $n = 3$ nearest healthy columns on each side:

$$U = \sum_{j \in \mathcal{N}(c)} w_j \cdot X_j, \qquad w_j = \frac{1/|j-c|}{\sum_{k \in \mathcal{N}(c)} 1/|k-c|} \tag{2}$$

Second, rather than applying a centred filter to U (which would blur step-edges), a 1D Side-Window Filter with radius $r = 8$ is applied. For each pixel i, two sub-windows are considered — the left half $W_L = [i-r,\, i]$ and the right half $W_R = [i,\, i+r]$ — and the sub-window whose mean is closest to U_i is selected:

$$\tilde{U}_i = \text{mean}\left(\underset{W \in \{W_L,\, W_R\}}{\arg\min} \left| \text{mean}(W) - U_i \right| \right) \tag{3}$$

This prevents smoothing from averaging across physical edges in the scene, which is critical when a defective column intersects a high-contrast region. Third, the smoothed guide $\tilde{U}$ is subtracted from the raw defective column to isolate high-frequency content $X_{\text{high}} = X_{\text{raw}} - \tilde{U}$, which contains both the stripe noise and fine image textures. Fourth, the Gradient Domain Guided Image Filter [2] fits a local linear model in a sliding window ω_k of height $h = 16$:

$$S_i = a_k \cdot \tilde{U}_i + b_k, \qquad \forall\, i \in \omega_k \tag{4}$$

The coefficients (a_k, b_k) are obtained by minimising:

$$E = \sum_{i \in \omega_k} \left[\left(a_k \tilde{U}_i + b_k - X_{\text{high},i} \right)^2 + \frac{\mu}{\Gamma_{\tilde{U}}} \left(a_k - \gamma_k \right)^2 \right] \tag{5}$$

where γ_k is an edge perception weight that reduces regularisation near edges and increases it in smooth regions. The regularisation parameter $\mu = 0.022$ follows the optimal value reported in [2]. Final coefficients are averaged over all overlapping windows, and the corrected column is obtained via Equation 1.

The method operates entirely in 1D along each defective column and requires no global image processing, making it well-suited for embedded or edge-computing deployment.

Method comparison and selection.

Table 3 summarises the quantitative and qualitative evaluation of all tested correction methods on the VGA training set. The Final Score is the challenge metric defined in the Introduction; the Max. Residual reports the peak absolute pixel error observed in the visual residual maps (Figure 9).

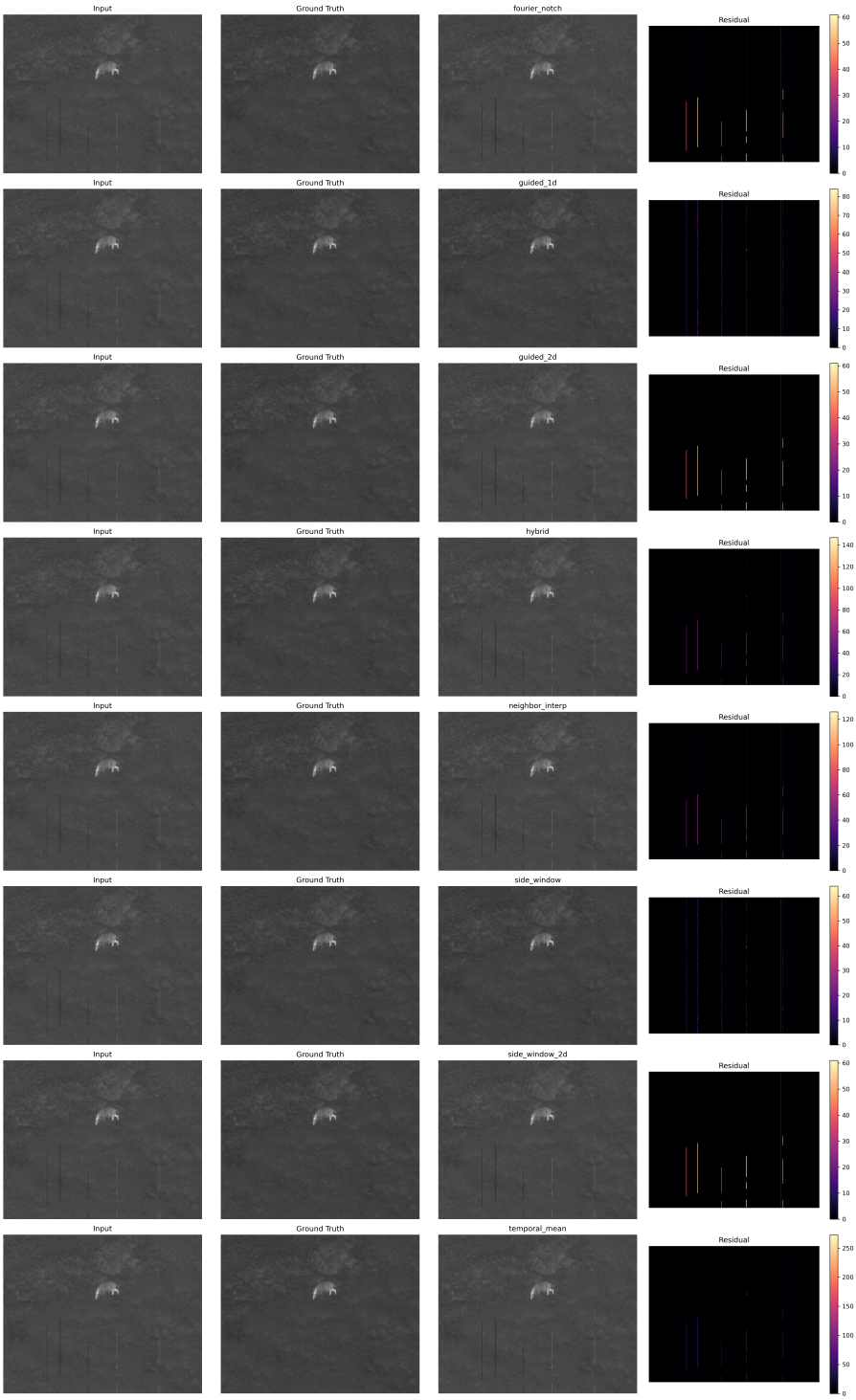

Figure 9: Visual comparison of all tested correction methods on SXGA sequence 1, simulation 3, frame 43. For each method: input image (left), ground truth (centre-left), corrected output (centre-right), and absolute residual map (right). Lower residual values indicate better correction quality.

Table 3: Comparison of correction methods on the VGA training set. Final Score is averaged over all sequences and simulation conditions. Max. Residual (LSB) is the peak value observed in the visual residual maps.

Method	Score ↑	Max Residual ↓	Key limitation
side_window	**0.7035**	∼40	— (selected)
guided_1d	0.6733	∼80	Slightly weaker edge preservation than side
fourier_notch	0.4779	∼60	Ringing artefacts; fails on non-periodic defects
guided_2d	0.4384	∼60	Vertical incoherence from pixel-wise guide construction
side_window_2d	0.4384	∼80	Vertical incoherence from pixel-wise guide construction
neighbor_interp	0.4601	∼100	Blind interpolation smears diagonal edges
hybrid	0.2836	∼140	Ghosting in dynamic scenes
temporal_mean	0.1974	∼250	Ghosting in dynamic scenes

The `side_window` method was selected as the final approach due to its consistently lowest residual and superior robustness across all sequence types. For latency-constrained applications, `guided_1d` offers a favourable trade-off, running approximately 30–40% faster while maintaining acceptable correction quality.

Abandoned approaches and failure analysis.

Reaching the final method required systematically testing and eliminating several candidate approaches. The visual residual map served as the primary diagnostic tool throughout this process.

2D spatial correction (`guided_2d`, `side_window_2d`). Given that the neural network produces a dense 2D pixel-level probability mask, it was natural to attempt 2D patch-based correction methods. However, empirical results showed a critical drop in performance, with F1-scores frequently dropping to near zero. The failure had two root causes. First, incorporating horizontal pixel variations into the local linear model prevented the GDGIF from isolating the strictly vertical stripe component: the filter could not distinguish horizontal image structure from vertical noise. Second, the pixel-wise guide construction introduced vertical incoherence, adjacent pixels in the same defective column received guides computed from slightly different local neighbourhoods, breaking the spatial consistency that the GDGIF linear model requires. The 1D formulation avoids both issues by computing a single consistent guide for the entire column.

Fourier notch filtering (`fourier_notch`). Columnar stripe noise concentrates at low spatial frequencies when analysed along the column axis, making frequency-domain filtering a theoretically attractive approach. In practice, the method performed competitively for constant noisy columns, but failed for blinking and fragmented defects, which violate the periodicity assumption implicit in Fourier analysis. Additionally, the hard frequency cutoff introduced Gibbs-phenomenon ringing artefacts near high-contrast edges. Since `side_window` matched `fourier_notch`'s best-case performance while generalising better across all defect types, the Fourier approach was not selected.

Temporal approaches (`temporal_mean`, `hybrid`). We hypothesised that replacing a defective column's value with its pixel-wise temporal median would cancel noise and blinking contributions over time. This held for static scenes, but failed in the dynamic sequences present in the dataset. When the camera or objects move between frames, averaging pixel values over time destroys the per-frame spatial structure, producing ghosting artefacts, dark vertical bands where moving objects occupied the defective column in different frames. The residual map confirmed this failure, with peak errors reaching 250 LSB.

Simple spatial interpolation (`neighbor_interp`). This baseline replaced defective values with a distance-weighted average of the nearest healthy columns. While computationally minimal, it acts as a blind low-pass filter in the horizontal direction: when a defective column intersects a diagonal scene feature, the interpolation smears the edge, introducing systematic localised errors that the GDGIF-based methods avoid through their stripe-specific decomposition.

3 Results

Our final version uses the detector 5: Local Mean of Medians, with the correction Side-Window Filter + 1D Gradient Domain Guided Image Filter. This gives us an average final score of 0.75 on the training data set. Complete results are presented in the following table (Table 4).

Table 4: Results of the final version of our algorithm

Sensor	Sequence	F1	RMSE def norm	RMSE ok norm	Final Score
HD	sequence 1.1	0.54	0.85	0.95	0.78
HD	sequence 1.2	0.26	0.84	0.94	0.68
HD	sequence 1.3	0.30	0.62	0.95	0.62
HD	sequence 2.1	0.74	0.76	0.99	0.83
HD	sequence 2.2	0.55	0.72	0.98	0.75
HD	sequence 2.3	0.45	0.73	0.98	0.72
HD	sequence 3.1	0.67	0.76	0.98	0.80
HD	sequence 3.2	0.46	0.63	0.98	0.69
HD	sequence 3.3	0.64	0.74	0.99	0.79
SXGA	sequence 1.1	0.80	0.75	0.99	0.85
SXGA	sequence 1.2	0.47	0.75	0.98	0.73
SXGA	sequence 1.3	0.86	0.74	0.99	0.86
SXGA	sequence 2.1	0.49	0.70	0.95	0.71
SXGA	sequence 2.2	0.17	0.63	0.93	0.58
SXGA	sequence 2.3	0.35	0.66	0.94	0.64
SXGA	sequence 3.1	0.60	0.73	0.98	0.77
SXGA	sequence 3.2	0.44	0.68	0.98	0.70
SXGA	sequence 3.3	0.52	0.67	0.98	0.72
VGA	sequence 1.1	0.73	0.76	0.99	0.83
VGA	sequence 1.2	0.81	0.75	1.00	0.85
VGA	sequence 1.3	0.90	0.67	1.00	0.86
VGA	sequence 2.1	0.86	0.75	1.00	0.87
VGA	sequence 2.2	0.60	0.74	0.99	0.77
VGA	sequence 2.3	0.84	0.74	1.00	0.86
VGA	sequence 3.1	0.63	0.76	0.98	0.79
VGA	sequence 3.2	0.16	0.66	0.97	0.59
VGA	sequence 3.3	0.78	0.71	0.99	0.83

4 Conclusion

This study addresses the challenge of detecting and correcting defective vertical columns in infrared image sequences. Five detection methods were systematically developed and evaluated, encompassing both simple statistical techniques and more advanced deep learning approaches.

Detector 5 (Local mean of medians) demonstrated the highest overall performance, achieving an average F1-score of 0.57 across VGA, SXGA, and HD sensors. This method effectively addressed consecutive defective columns by comparing each column's median to a locally smoothed reference, and significantly outperformed earlier approaches such as neighbour differencing and temporal standard deviation.

For correction, a dedicated 1D pipeline was developed based on a Side-Window Filter combined with a 1D Gradient Domain Guided Image Filter. This approach conceptualises correction as a signal decomposition problem, estimating and subtracting only the additive stripe noise component while leaving

all healthy pixels untouched. The method achieved the highest correction score (0.7035 on the VGA training set) while preserving sharp edges and fine image details. Notably, it operates entirely in 1D along each defective column, making it computationally efficient and well-suited for embedded or real-time applications.

Although deep learning-based detectors, particularly the 2D asymmetric CNN, showed strong potential with F1-scores reaching 0.89 under earlier metric versions, their performance was sensitive to changes in the evaluation protocol. In contrast, the final statistical detection combined with the 1D correction provided a strong, stable solution over various sensors and defect types.

To summarise, the combination of Detector 5 with the Side-Window and Gradient Domain Guided filtering pipeline provides an effective, efficient, and feasible approach for identifying and restoring defective columns in infrared imagery. The findings show that carefully designed 1D signal analysis methods, when adapted to the specific morphology of vertical stripe defects, can outperform more complex alternatives in terms of both accuracy and deployability.

References

[1] Z. He, Z. Chen, G. Fu, Y. Zheng, and Z.-M. Lu, "An edge-preserving stripe noise removal method for infrared images," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 2, pp. 169–172, 2025.

[2] Y. Shao, Y. Sun, M. Zhao, Y. Chang, Z. Zheng, C. Tian, and Y. Zhang, "Infrared image stripe noise removing using least squares and gradient domain guided filtering," *Infrared Physics & Technology*, vol. 119, p. 103968, 2021.